CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: LendIt

Prepared By: Quirante, Jazpher Neil P.

Version: 2.0

Date: 2/27/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	2/21/2026	Jazpher Neil p. Quirante	Introduction, Functional & Non-Functional Requirements, System Architecture	Done
2.0	2/28/2026	Jazpher Neil p. Quirante	API Contracts/ Communication, Database Design, UI/UX, Plan	Done

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

LendIt is a cross-platform item lending system designed for students. It enables students to lend and borrow items within their campus community through a React web application, an Android mobile application built with Kotlin, and a Spring Boot backend supported by a relational database. The system allows users to list items, browse available listings, send borrow requests, communicate through in-app messaging, and manage lending transactions across web and mobile platforms.

1.2 Objectives

1. Develop a fully functional sharing and lending system with authentication, item listing, borrowing requests, and transaction management.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android application developed using Kotlin (mobile).
3. Develop RESTful APIs to enable smooth communication between the web, mobile, and backend components.
4. Provide secure access using JWT authentication, BCrypt password hashing, and protected routes
5. Design a consistent and user-friendly interface for both web and mobile platforms
6. Deploy the system in a production-ready environment to ensure reliability and scalability.

1.3 Scope

Included Features:

- User registration, authentication (email/password) and password recovery
- JWT-based authentication with BCrypt password hashing
- Role-based access control with Student (Borrower/Lender) and Admin roles
- Item listing with search and filter functionality
- Borrow request management (request, approve, reject)
- Lending and borrowing transaction tracking
- Simple in-app messaging between lenders and borrowers

- File upload for item images stored on the server and linked to item records
- Email notifications
- Administrative tools for managing users, items, and reports
- Responsive interface for both web and mobile platform
- Relational database to store users, items, requests, and transaction records

Excluded Features:

- Real payment gateway integration
- User rating and feedback system for items or borrowers
- SMS notifications
- Social media integration (e.g., Facebook, Instagram)
- GPS/Location tracking

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the LendIt system. It outlines the system requirements, architecture, API structure, database design, and integration features to guide development and ensure seamless interaction between the web, mobile, and backend components.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: LendIt
Domain: Lending Platform
Primary Users:

1. Students (Lenders and Borrowers)
2. Administrators

Problem Statement: Students often own items that are rarely used, while other students may temporarily need those same items. However, there is no simple and centralized system that facilitates safe and organized lending within a student

community.
Solution: LendIt is a platform where students can list items, browse available listings, request to borrow, and manage lending transactions across web and mobile apps.

2.2 Core User Journeys

Journey 1: New User Using the System

1. User opens the web or mobile application
2. Register a new account
3. Logs in to access the system
4. Browse available items listed by other students
5. Views item details and availability
6. Sends a borrow request to the item owner
7. Waits for approval and receives confirmation once accepted
8. Borrows the item and later marks it as returned

Journey 2: User Borrowing an Item

1. User logs in with existing credentials
2. Searches for specific item
3. Opens the item details page
4. Sends a borrow request to the item owner
5. Receives approval notification
6. Communicates with the lender via in-app messaging
7. Confirms return once the item is given back

Journey 3: User Lending an Item

1. Student logs in using the same account
2. Navigates to “My Items”
3. Adds a new item with pictures, description and availability
4. Receives borrow requests from other students
5. Reviews and approves or rejects requests
6. Tracks the lending status until the item is returned

Journey 4: Administrator Oversight

1. Admin logs in with elevated privileges
2. Views all users and listed items
3. Monitors borrow transactions and reports
4. Removes inappropriate or invalid listings

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout, forgot password)
2. User Profile Management
3. Item catalog (list, view details, search)
4. Borrow request management (request, approve, reject)
5. Lending transaction tracking
6. Basic messaging between lender and borrower
7. File upload for item images
8. Email notification
9. Admin panel (product CRUD, order view)

SHOULD HAVE

1. Item categories and filtering options
2. Borrow and lending history tracking
3. Settings page (enable notification, dark mode etc..)

COULD HAVE

1. Due date reminders
2. Bookmarking or saving favorite items
3. Item reservation or booking before approval

WON'T HAVE

1. Real payment processing in production (sandbox only if applicable)
2. Social media sharing features
3. Advanced analytics dashboards
4. Multi-language localization

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/forgot-password, GET /me
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Item Catalog

- **Screens:** Item Listing, Item Detail
- **Display:** Grid layout showing item images, names, descriptions, and availability status
- **Search:** By item name or keyword
- **Search:** By product name
- **API Endpoints:** GET /items, GET /items/{id}, GET /items/search, DELETE /items/{id
- **Admin Functions:** POST /items, PUT /items/{id}, DELETE /items/{id}

Feature: Borrow Request Management

- **Screens:** Borrow Request Form, Request Status List
- **Functions:** Send borrow request, cancel request, approve or reject request
- **Persistence:** Stored in database per user and linked to items
- **API Endpoints:** POST /requests, GET /requests/user, PUT/requests /{id}/approve, PUT /requests/{id}/reject, DELETE /requests/{id}

Feature: Lending Transaction Tracking

- **Screens:** Active Borrows, Lending History
- **Data Collected:** Borrower, lender, item, borrow date, return date, status
- **Process:** Create transaction when request is approved and update status when item is returned
- **API Endpoints:** POST /transactions, GET /transactions/user, PUT /transactions/{id}/complete

Feature: In-App Messaging

- **Screens:** Chat Interface between borrower and lender
- **Functions:** Send and receive messages related to a borrow request
- **Persistence:** Messages stored and linked to a specific borrow request
- **API Endpoints:** GET /messages/{requestId}, POST /messages

Feature: User Profile

- **Screens:** View, Edit Profile

- **Functions:** Update name, email, profile picture, and contact information
- **API Endpoints:**GET /users/profile, PUT /users/profile

2.5 Acceptance Criteria

AC-1: Successful User Registration

Given I am a new user
When I enter a valid email and strong password
And confirm the password correctly
And click "Create Account"
Then my account should be created
And I should be automatically logged in
And redirected to the homepage

AC-2: Borrow Request Flow

Given I am logged in as a student
When I view an available item
And click "Request to Borrow"
Then a borrow request should be sent to the item owner
And the request status should be visible in my request list

AC-3: Approve Borrow Request

Given I am the item owner
When I receive a borrow request
And click "Approve"
Then a lending transaction should be created
And the item status should change to "Borrowed"

AC-4: Return Item

Given a borrowing transaction is active
When the borrower marks the item as returned
Then the transaction status should be updated to "Completed"
And the item should become available again

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on standard broadband

- Mobile app cold start: ≤ 3 seconds
- Support at least 100 concurrent users

- Database queries should complete within 500 ms
- Image uploads limited to 5MB per file

3.2 Security Requirements

- HTTPS encryption for all communications
- JWT-based authentication with secure refresh tokens
- Password hashing using BCrypt (salt rounds = 12)
- Role-based authorization for Admin and Users
- Rate limiting: 100 requests per minute per IP
- Secure file upload validation for item images
- Protected API routes using JWT
- Role-based authorization enforcement

3.3 Compatibility Requirements

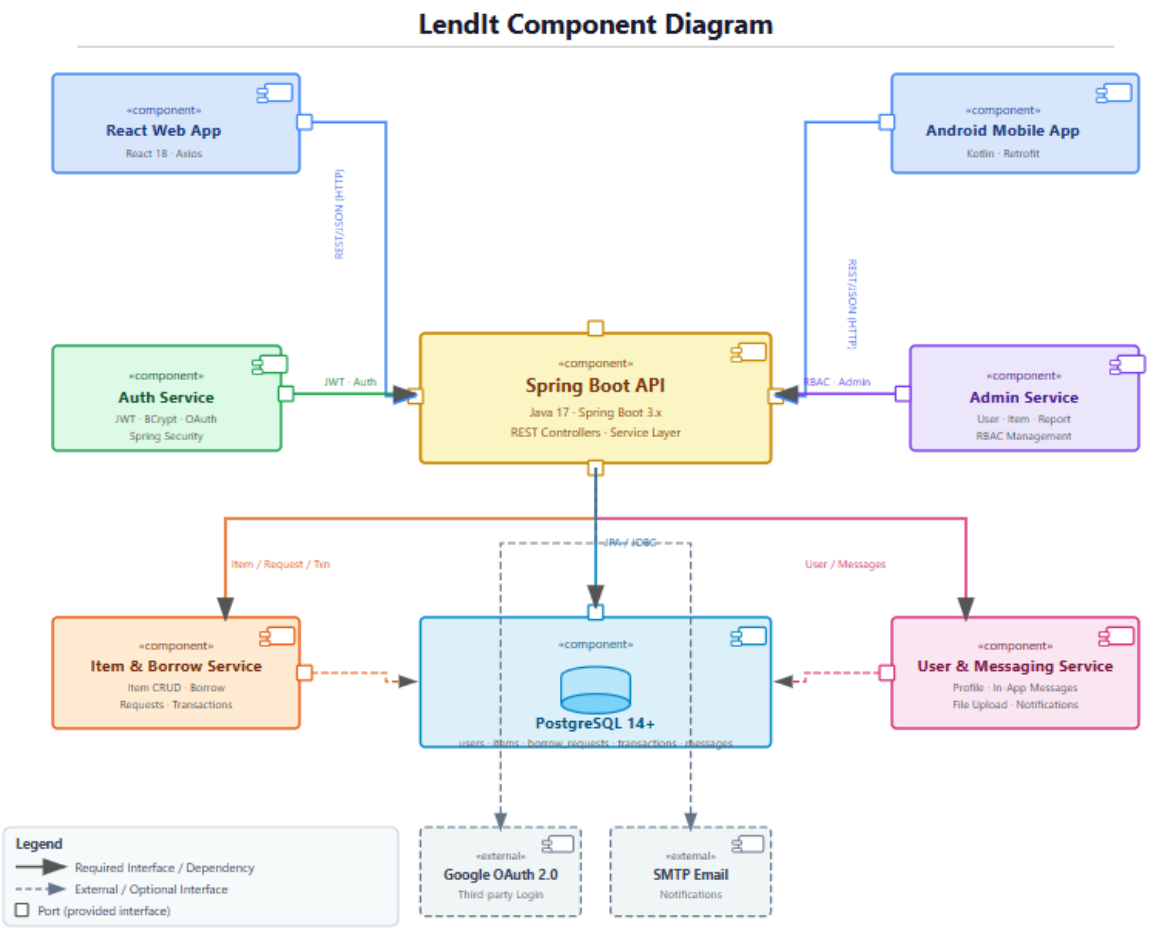
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

- New users should be able to request an item within 5 minutes
- Consistent navigation across web and mobile platforms
- Clear validation and error messages with recovery suggestions
- Simple and intuitive UI for users with minimal learning curve

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18,, Axios
- **Mobile:** Kotlin,
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Vercel/Render (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	http://localhost:8080/api
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	api/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"fName": "<firstname>",
	"lName": "<lastname>",
	"phone": "<phone>",
	"email": "<email>",
	"password": "<password>"
	}

Response Structure	{
	"success": boolean,
	"data": {
	user {
	"id": <id>,
	"fName": "<firstname>",
	"lName": "<lastname>",
	"phone": "<phone>",
	“email”: <email>,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null

	}, "timestamp": string }

User Login

Description	User Login
API URL	api/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	"id": <id>,
	"fName": "<firstname>",
	"lName": "<lastname>",
	"phone": "<phone>",
	“email”: <email>,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Logout

Description	User Logout
API URL	api/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Session-based
Request Payload	None
Response Structure	{

	"success": boolean,
	“message”: string,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	}

Get Current User Profile

Description	Retrieves the currently authenticated user's profile using JWT.
API URL	/api/user/me
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT) required

Request Payload	{
	Authorization: Bearer <jwt_token>
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	"id": <id>,
	"fName": "<firstname>",
	"lName": "<lastname>",
	"phone": "<phone>",
	“email”: <email>,
	“password”: null
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	}

Item Management Endpoints

Get All Items

Description	Retrieves all available items for lending.
API URL	/api/items
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"itemId": <item_id>,
	"name": "<name>",
	"description": "<description>",
	"imageUrl": "<url>",
	"availability": "AVAILABLE",
	"ownerId": <owner id>
	}
	],
	"error": null,
	}

Get Item by ID

Description	Retrieves detailed information of a specific item.
API URL	api/items/{id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": true, "data": [{ ""itemId": <item_id>, "name": "<name>", "description": "<description>", "imageUrl": "<url>", "availability": "AVAILABLE",

	"ownerId": <owner id> }], "error": null, }

Create Item Listing

Description	Allows a user to list a new item for lending.
API URL	api/items/add
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token required

Request Payload	{
	"name": "<item name>",
	"description": "<description>",
	"imageUrl": "<url>"
	}

Response Structure	{
	"success": true,
	"data": {
	"itemId": <item_id>,
	"name": "<item name>",
	"description": "<description>",
	"availability": "AVAILABLE"
	},
	"error": null,
	}

Update Item

Description	Updates details of an existing item.
API URL	api/items/{id}
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token required

Request Payload	{
	"name": "<updated name>",
	"description": "<updated description>",
	"availability": "AVAILABLE"
	}

Response Structure	{
	"success": true,
	"data": {
	"itemId": <item_id>,
	"name": "<updated name>",
	"description": "<updated description>",
	"availability": "AVAILABLE"
	},
	"error": null,
	}

Delete Item

Description	Deletes an item listing.
API URL	api/items/delete/{id}
HTTP Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer Token required
Request Payload	None

Response Structure	{
	"success": true,
	“message”: “<Success message>”
	"error": null,
	}

Borrow Request Endpoints

Send Borrow Request

Description	Allows a borrower to request an item.
API URL	/api/requests
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"itemId": <item_id>,
	"message": "<Message>"
	}

Response Structure	{
	"success": true,
	"data": [
	{
	"requestId": <id>,
	"status": "PENDING"
	],
	"error": null,
	}

Get User Request

Description	Retrieves all borrow requests made by the current user.
API URL	/api/requests/{id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"requestId": “<request_id>”,
	"itemId": “<item_id>”,
	"status": "PENDING"
	],
	"error": null,
	}

Approve Borrow Request

Description	Allows item owner to approve a borrow request.
API URL	/api/requests/{id}/approve
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": {
	"requestId": <id>,
	"status": "APPROVED"
	},

	"error": null, }

Reject Borrow Request

Description	Allows item owner to reject a borrow request.
API URL	/api/requests/{id}/reject
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": {
	"requestId": <id>,
	"status": "REJECTED"
	},
	"error": null,
	}

Transaction Endpoints

Create Transaction

Description	Creates a lending transaction when a request is approved.
API URL	/api/transactions
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"transactionId": <id>,
	"status": "ACTIVE"
	],
	"error": null,
	}

Complete Transaction

Description	Marks a transaction as completed when item is returned.
API URL	/api/transactions/{id}/complete
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"transactionId": <id>,
	"status": "COMPLETED"
	],
	"error": null,
	}

Messaging Endpoints

Get Messages by Request

Description	Retrieves chat messages related to a borrow request.
API URL	/api/messages/{requestId}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"senderId": “<sender_id>”,
	"content": "<message>”,
	"timestamp": "2026-02-21T10:30:00
	Z"
	],
	"error": null,
	}

Send Message

Description	Sends a message between borrower and lender.
API URL	/api/messages
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"requestId": <request_id>,
	"content": <message>
	}

Response Structure	{
	  "success": true,
	  "data": [
	  {
	  "messageId": <id>,
	  "status": "SENT"
	  ],
	  "error": null,
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	json
	{
	  "success": false,
	  "data": null,

	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

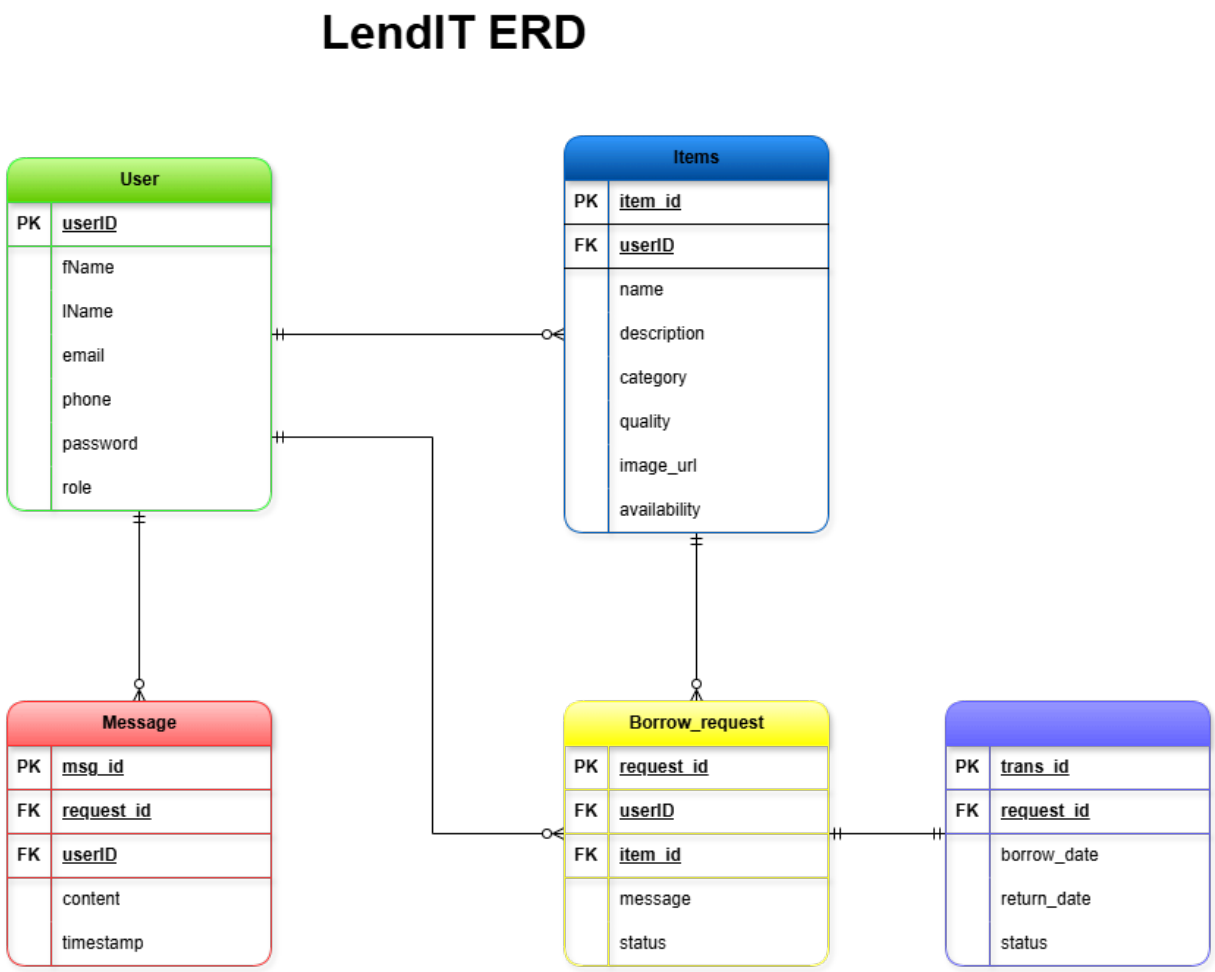
Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many:** User → Items (A user can list multiple items)
- **One-to-Many:** User → Borrow Requests (A user can send multiple requests)
- **One-to-Many:** Item → Borrow Requests (One item can receive multiple requests)
- **One-to-One: Borrow** Request → Transaction (Created when request is approved)
- **One-to-Many:** Borrow Request → Messages (Chat messages per request)

Key Tables:

1. **users** - User accounts and authentication
2. **items** - Stores items listed by users for lending.
3. **borrow_requests** - Stores borrowing requests made by users.
4. **transactions** - Tracks active and completed lending transactions
5. **messages** - Stores chat messages between borrower and lender for each request.

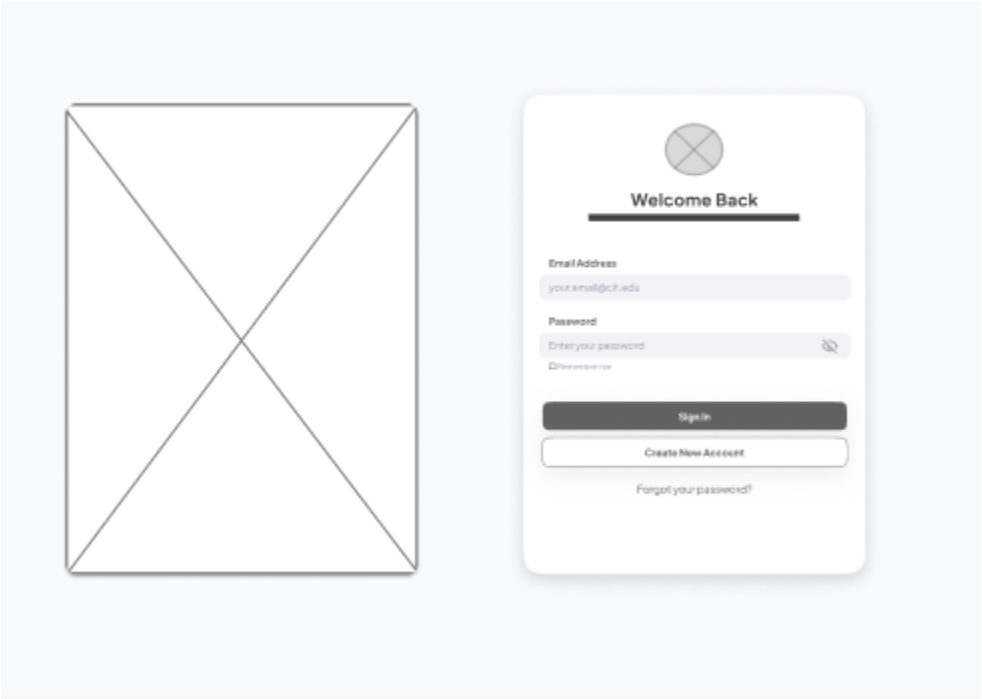
Table Structure Summary:

- **users:** user_id, fname, lname, email, phone, password, role
- **items:** item_id, name, description, category, quality, image_url, availability, owner_id
- **borrow_requests:** request_id, item_id, userID, message, status
- **transactions:** trans_id, request_id, borrow_date, return_date, status
- **messages:** msg_id, request_id, sender_id, content, timestamp

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

<u>Login Page</u>



Header: No navigation bar is displayed to keep the user focused on signing in.

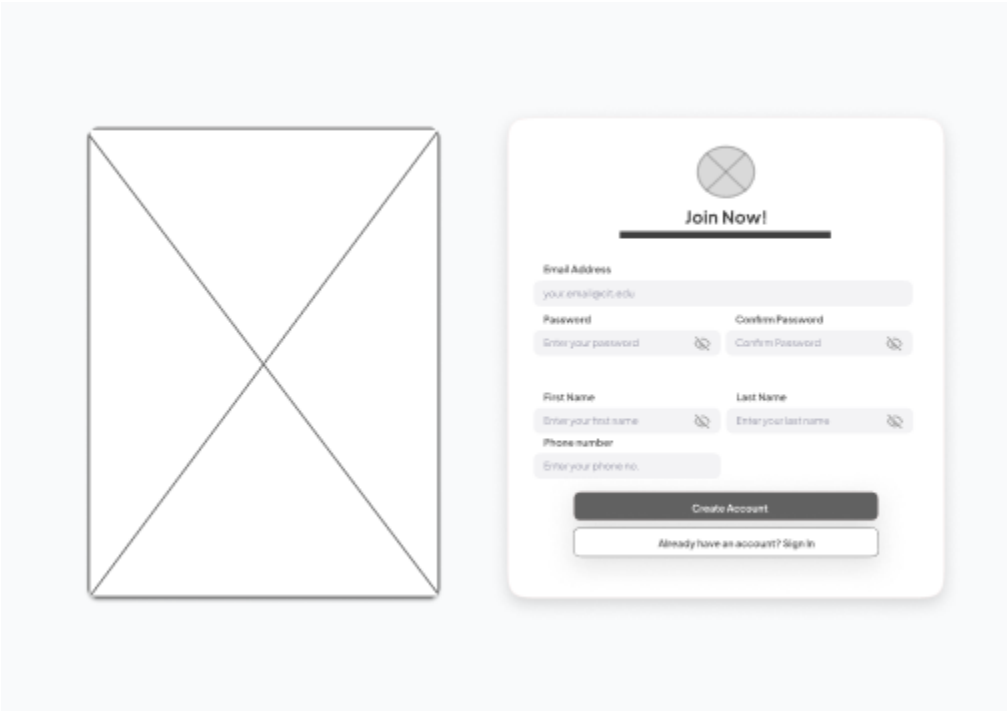
Content: Split layout with two main sections:

- Left panel reserved for an illustration
- Right panel contains the authentication card.

Login Card Components:

- Logo Placeholder at the top for system branding
- Title: “Welcome Back”
- Divider line below the title for visual separation
- Email Address Input Field
- Password Input Field with visibility toggle
- “Sign In” Button (Primary Call-To-Action)
- “Create New Account” Button (Secondary action below the primary button)
- “Forgot your password?” Link for account recovery

<u>Register Page</u>



Header: No navigation bar is displayed to keep the user focused on signing in.

Content: Split layout with two main sections:

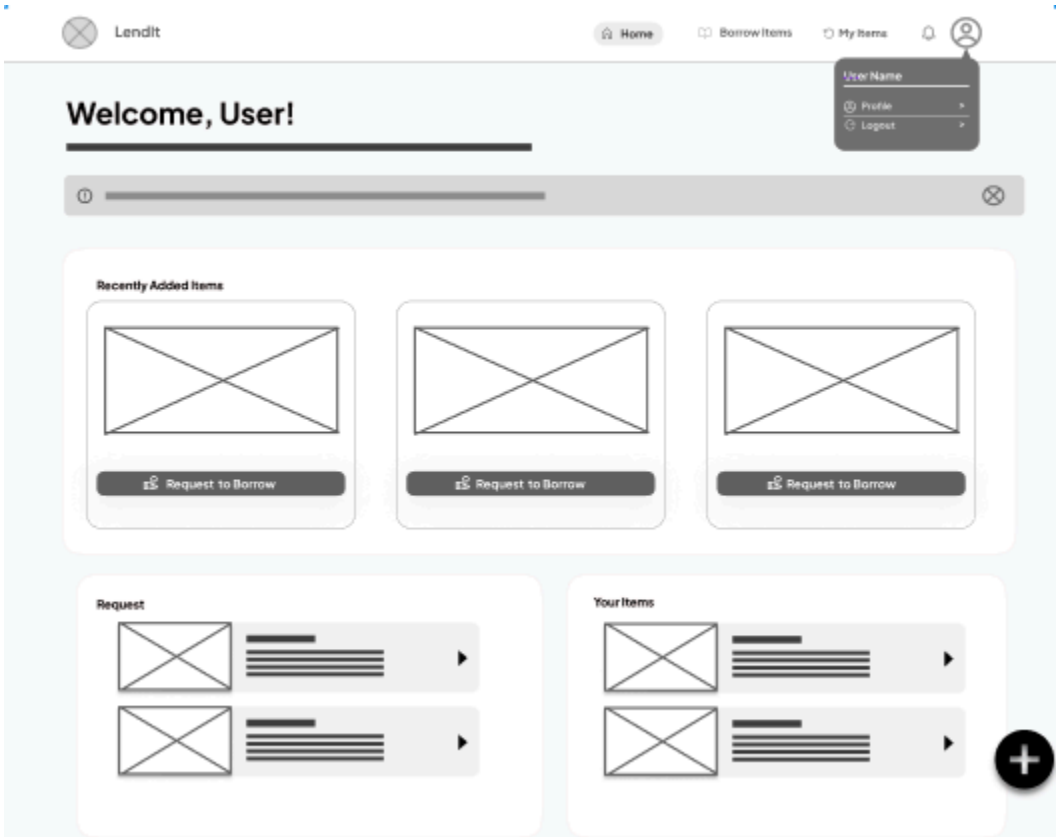
- Left panel reserved for an illustration

- Right panel contains the authentication card.

Registration Card Components:

- Logo Placeholder at the top for system branding
- Title: “Join Now!”
- Divider line below the title for visual separation
- Email Address Input Field
- Password Input Field with visibility toggle
- Confirm Password Input Field with visibility toggle
- First Name Input Field
- Last Name Input Field
- Phone Number Input Field
- “Create Account” Button (Primary Call-To-Action)
- “Already have an account? Sign In” Button/Link (Secondary action for returning users)

<u>Homepage</u>



Header: Top navigation bar with system logo (LendIt), menu links (Home, Borrow Items, My Items), and user profile dropdown (Profile, Logout).

Content: Main dashboard showing a welcome message and key item sections.

Recently Added Items:

- Row of item cards with image, brief info, and “Request to Borrow” button

Request Section:

- List of recent borrow requests with thumbnail, short details, and navigation arrow

Request Section:

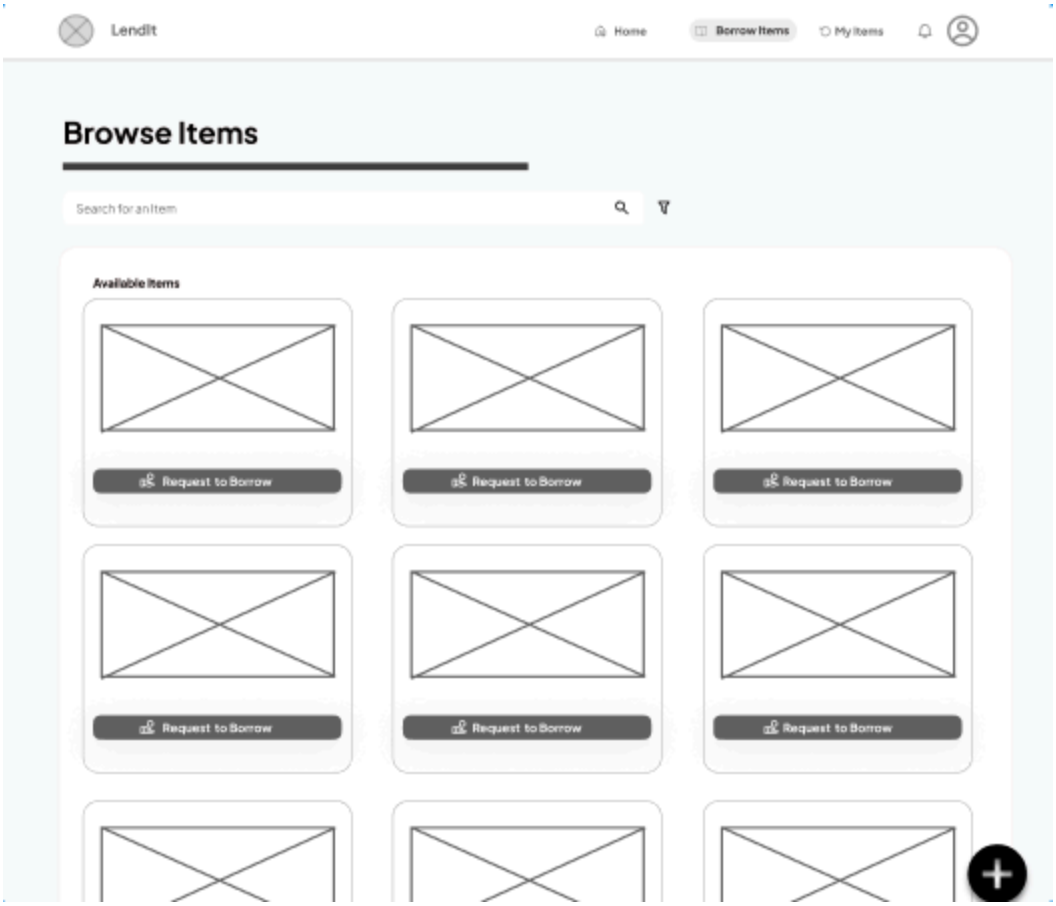
- List of recent borrow requests with thumbnail, short details, and navigation arrow

Your Items Section:

- List of user-owned items with image, description, and navigation arrow

Floating Action Button: “+” button at bottom-right for adding a new item.

<u>Browse Items Page</u>



Header: Top navigation bar with logo, menu links (Home, Borrow Items, My Items), and user profile icon.
Content: Main browsing interface for viewing all available items.
Browse Section:

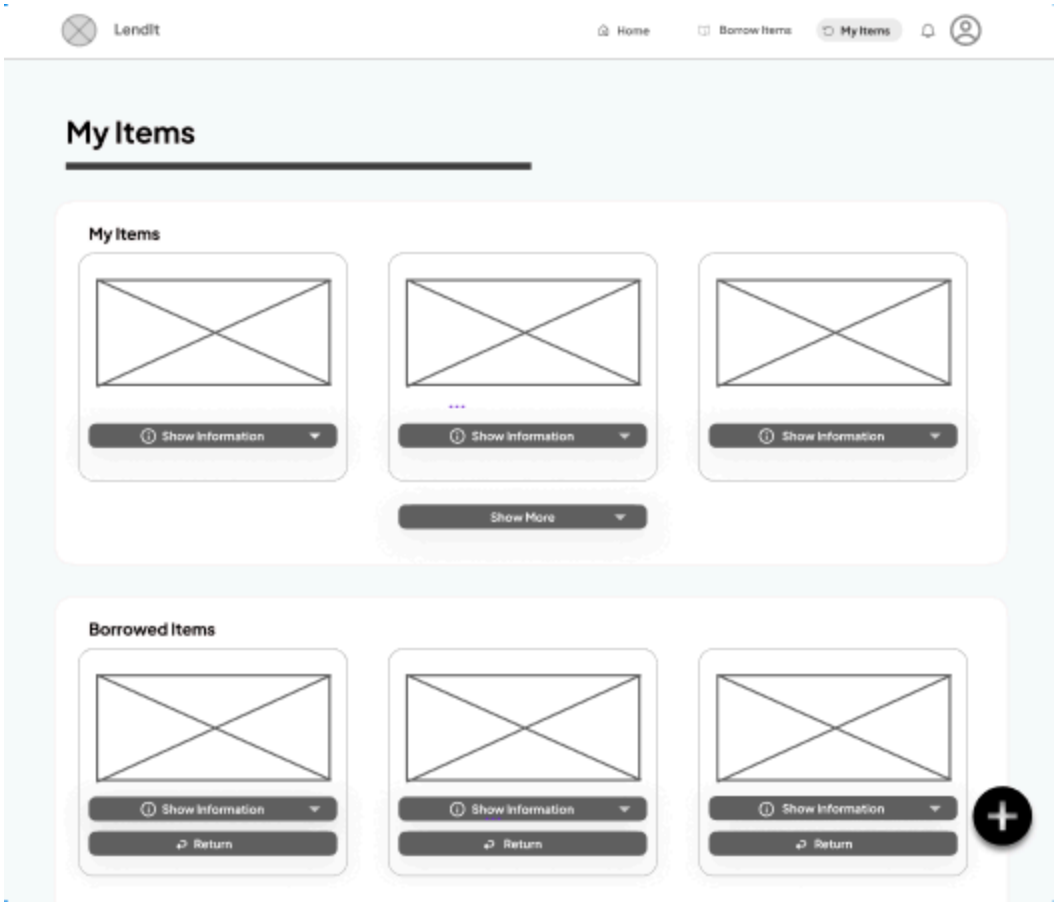
- Page title: “Browse Items”
- Search bar for finding specific items
- Filter and search icons for refining results

Available Items Grid:

- Grid layout of item cards
- Each card includes:
    - Item image placeholder
    - Brief item info area
    - “Request to Borrow” button for quick action

Floating Action Button: “+” button at bottom-right for adding a new item.

<u>My Items Page</u>



Header: Top navigation bar with system logo (LendIt), menu links (Home, Borrow Items, My Items), and user profile icon.

Content: Page displaying items owned by the user and items they have borrowed.
My Items Section:

- Grid of item cards representing user-listed items
- Each card includes:
    - Item image placeholder
    - Brief item details
    - “Show Information” dropdown/button for viewing more details
- “Show More” button to load additional items

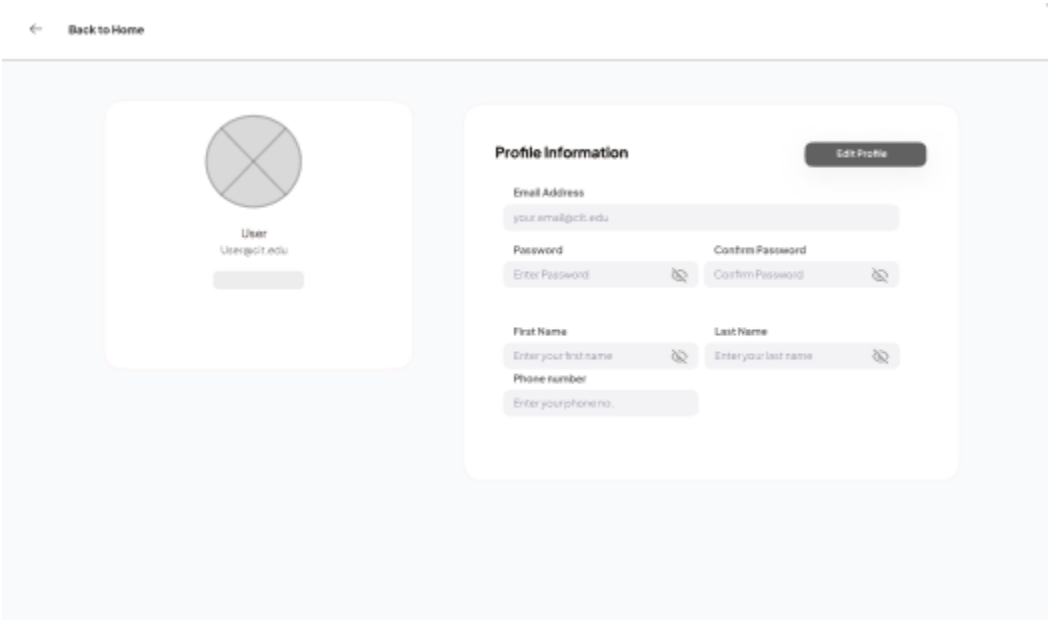
Borrowed Items Section:

- Grid of borrowed item cards
- Each card includes:
    - Item image placeholder
    - Brief item details
    - “Show Information” button for item details

- “Return” button to initiate item return process

Floating Action Button: “+” button at bottom-right for adding a new item.

Profile Page



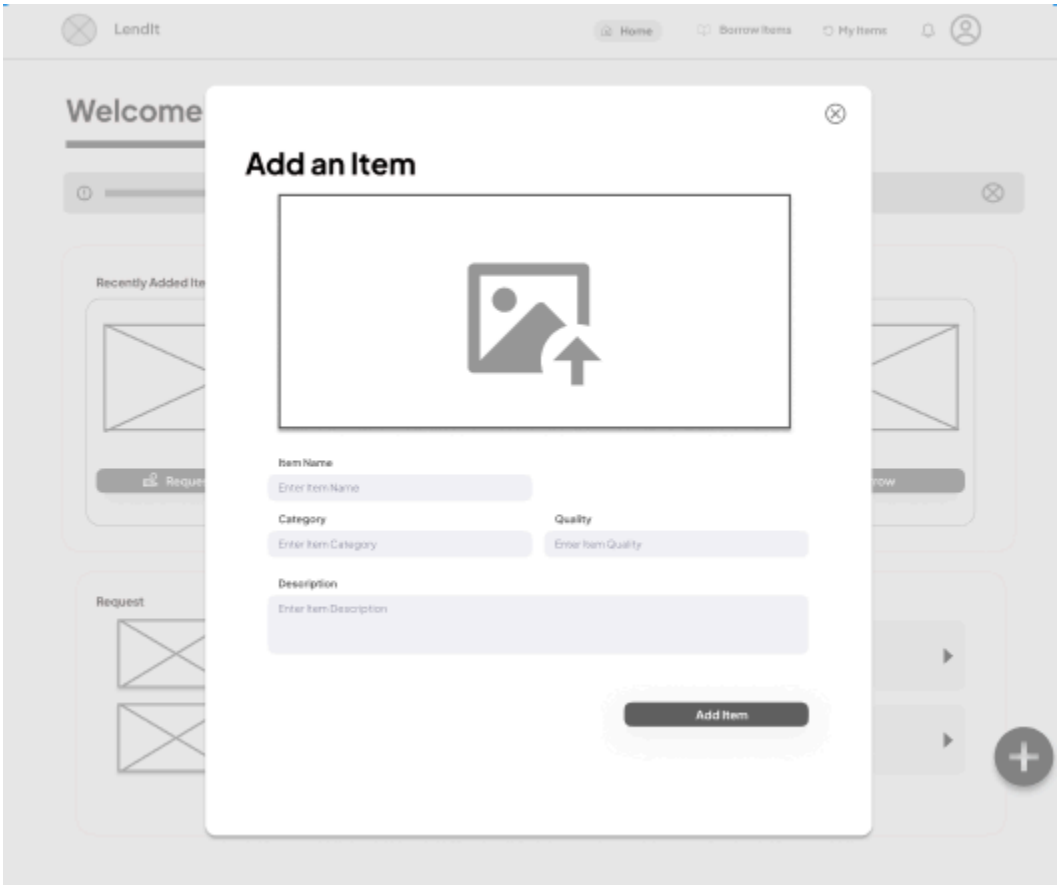
Header: Back button to return to the Home/Dashboard page.
Content: Split layout showing user overview and editable profile details.
User Overview Panel:

- Profile avatar placeholder
- Display name (User)
- Username or email preview

Profile Information Section:

- Title: “Profile Information”
- Email Address Field
- Password and Confirm Password Fields with visibility toggles
- First Name and Last Name Fields
- Phone Number Field
- “Edit Profile” Button to enable or save profile changes

<u>Add Item Popup</u>



Header: Modal popup with title “Add an Item” and a close (X) button at the top-right corner.
Content: Form modal overlay centered on the screen for adding a new item.
Image Upload Section:

- Large image upload placeholder
- Allows users to upload or drag-and-drop an item photo

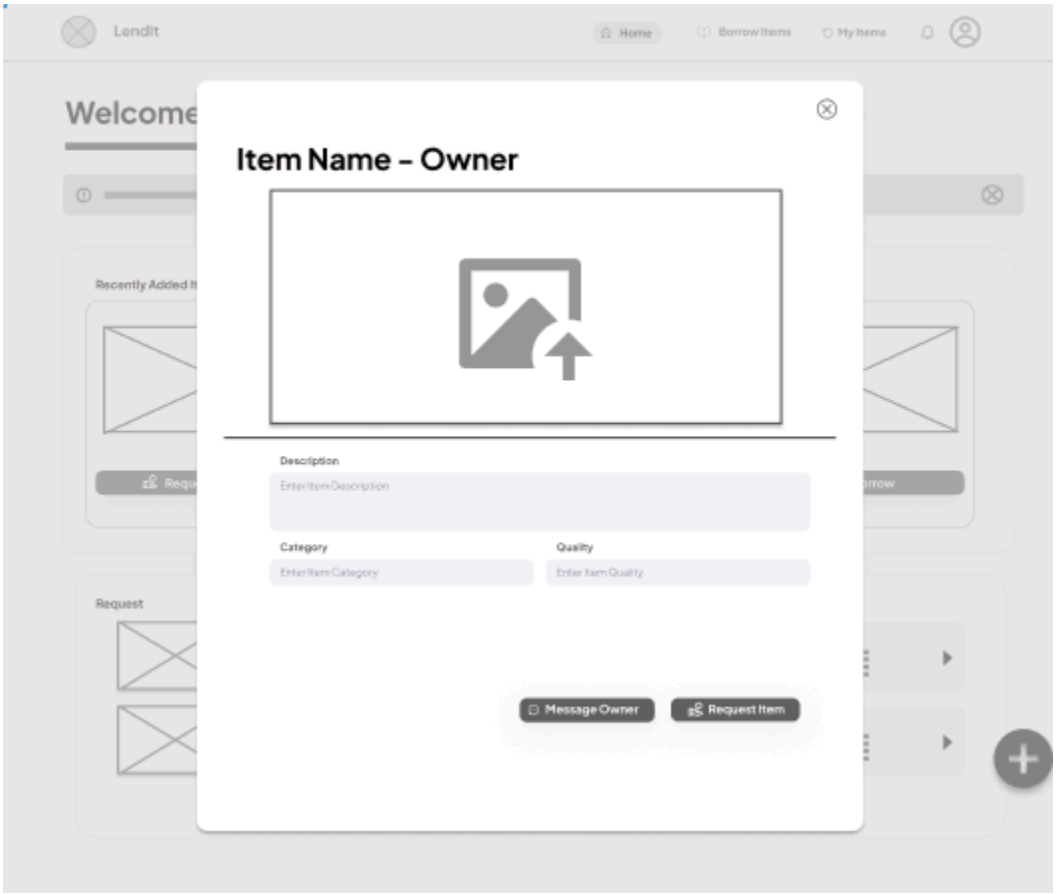
Form Fields:

- Item Name Input Field
- Category Input Field
- Quality Input Field
- Description Text Area for detailed item information

Action Button:

- “Add Item” Button to submit and create the new item listing

<u>Request Item Popup</u>



Header: Modal popup displaying the item title and owner name (e.g., “Item Name – Owner”) with a close (X) button at the top-right corner.
Content: Centered modal overlay presenting item details and request actions.
Item Preview Section:

- Large image placeholder showing the selected item

Item Details:

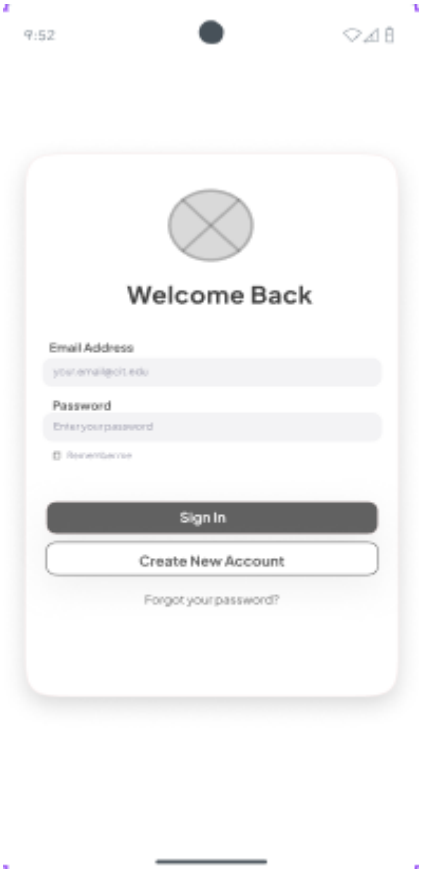
- Description text area showing item information
- Category field displaying item classification
- Quality field indicating item condition

Action Buttons:

- “Message Owner” Button to contact the item owner
- “Request Item” Button to submit a borrow request

7.2 Mobile Application Wireframes

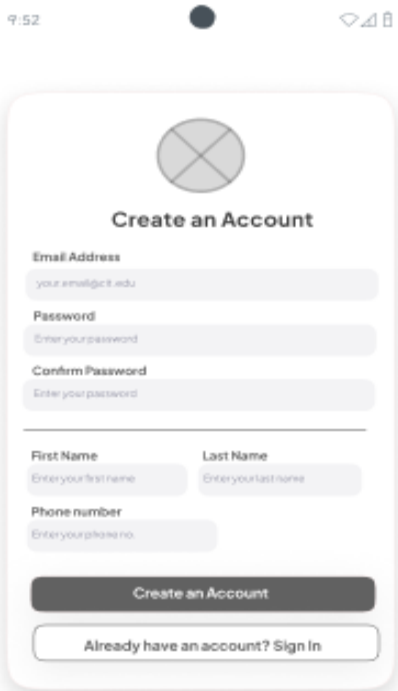
<u>Login Page</u>



Header: Minimal mobile status bar with no navigation to keep focus on authentication.
Content: Centered login card optimized for mobile screen size.
Login Card Components:

- Logo/Icon Placeholder at the top
- Title: “Welcome Back”
- Email Address Input Field
- Password Input Field
- “Remember me” checkbox option
- “Sign In” Button (Primary Call-To-Action)
- “Create New Account” Button (Secondary action)
- “Forgot your password?” Link for account recovery

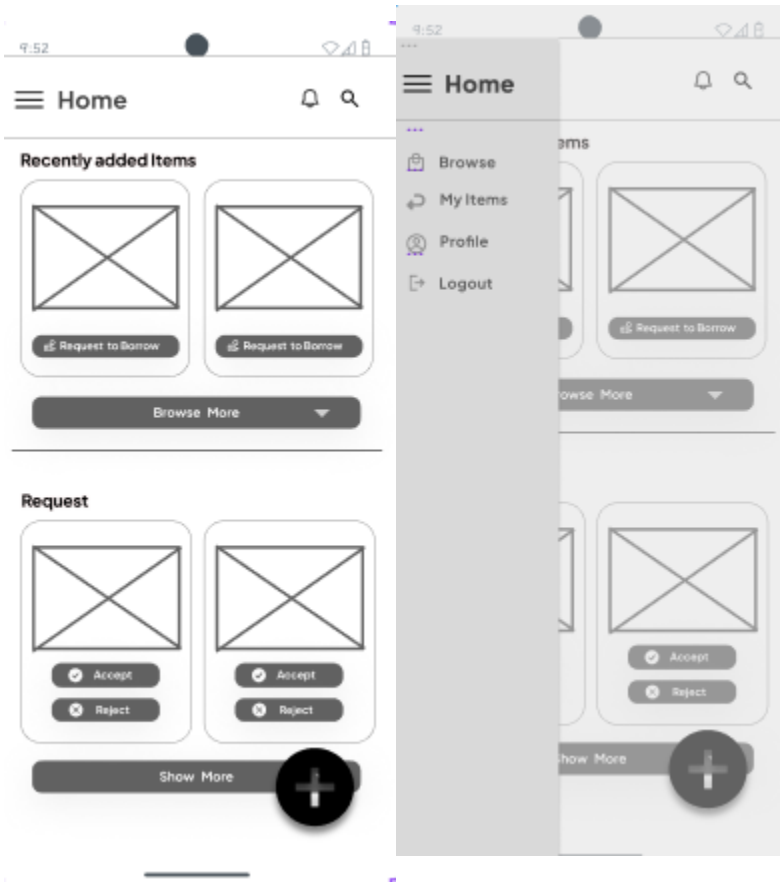
Register Page



Header: Minimal mobile status bar with no navigation to keep the user focused on account creation.
Content: Centered registration card in a single-column mobile layout.
Registration Card Components:

- Logo/Icon Placeholder at the top
- Title: "Create an Account"
- Email Address Input Field
- Password Input Field
- Confirm Password Input Field
- First Name and Last Name Input Fields (stacked for mobile)
- Phone Number Input Field
- "Create an Account" Button (Primary Call-To-Action)
- "Already have an account? Sign In" Button/Link for existing users

Homepage



Header: Top bar with hamburger menu icon, page title "Home," and action icons (notifications and search).
Navigation Drawer: Slide-out side menu containing links to Browse, My Items, Profile, and Logout.
Content: Scrollable dashboard layout displaying key sections for quick access.
Recently Added Items:

- Horizontal list of item cards with image placeholders

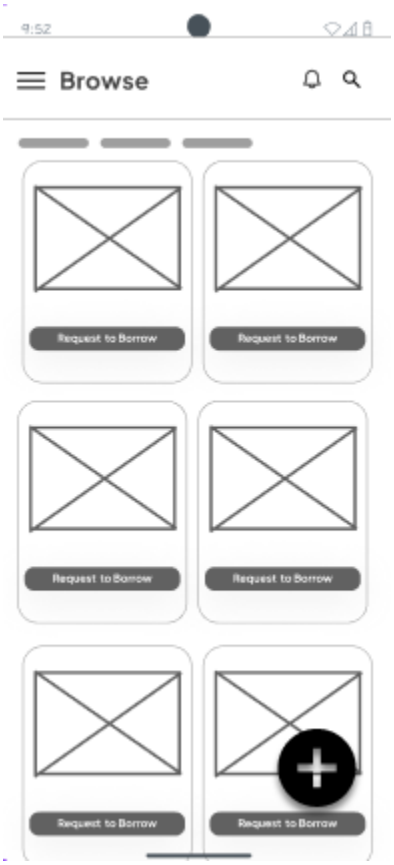
- Each card includes a “Request to Borrow” button
- “Browse More” dropdown/button to view additional items

Request Section:

- Grid/list of pending item requests
- Each request card includes item image and action buttons: “Accept” and “Reject”
- “Show More” button to load more requests

Floating Action Button: Circular “+” button at the bottom-right for adding a new item.

<u>Browse Item</u>

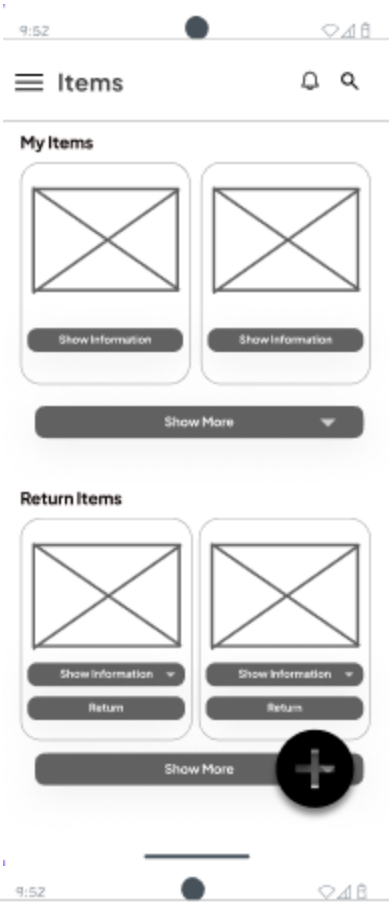


Header: Top bar with hamburger menu icon, title “Browse,” and action icons (notifications and search).
Content: Scrollable grid layout displaying all available items for borrowing.
Item Grid:

- Two-column grid of item cards
- Each card includes:
    - Item image placeholder
    - “Request to Borrow” button for quick action

Navigation Drawer: Accessible via hamburger menu with links to Home, Browse, My Items, Profile, and Logout.
Floating Action Button: Circular “+” button at the bottom-right for adding a new item.

My Items Page



Header: Top bar with hamburger menu icon, title "Items," and action icons (notifications and search).
Content: Scrollable layout showing user-owned items and borrowed items.
My Items Section:

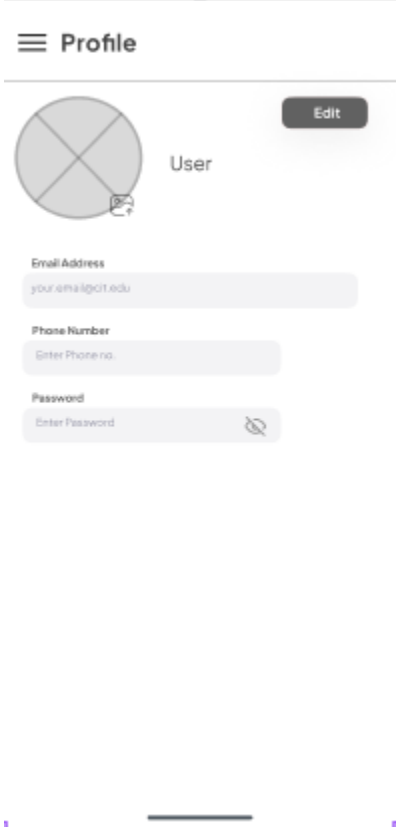
- Grid of item cards owned by the user
- Each card includes an image placeholder and "Show Information" button
- "Show More" button to load additional items

Return Items Section:

- Grid of borrowed item cards
- Each card includes an image placeholder, "Show Information" button, and "Return" button
- "Show More" button to display more borrowed items

Navigation Drawer: Accessible via hamburger menu with links to Home, Browse, My Items, Profile, and Logout.
Floating Action Button: Circular "+" button at the bottom-right for adding a new item.

Profile Page



Header: Top bar with hamburger menu icon and title "Profile."
Content: Scrollable profile management layout displaying user information.
Profile Overview:

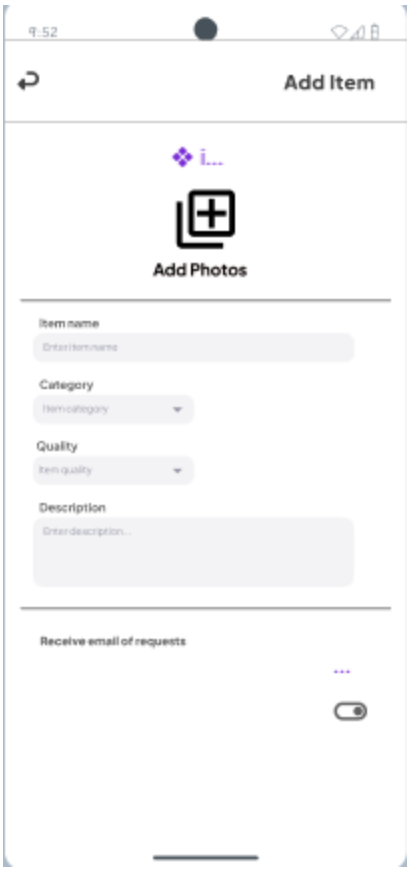
- Large profile avatar placeholder with edit icon
- Display name (User)
- "Edit" button to enable profile editing

Profile Information Fields:

- Email Address Field
- Phone Number Field
- Password Field with visibility toggle

Navigation Drawer: Accessible via hamburger menu with links to Home, Browse, My Items, Profile, and Logout.

<u>Add Item</u>



Header: Top bar with back arrow on the left and title "Add Item."
Content: Scrollable form layout for creating a new item listing.
Photo Upload Section:

- "Add Photos" area with upload icon
- Allows users to select or capture item images

Form Fields:

- Item Name Input Field
- Category Dropdown Selector
- Quality Dropdown Selector
- Description Text Area for item details

Notification Option:

- Toggle switch for "Receive email of requests" to enable or disable email notifications

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1–2: Finalize system requirements based on IT342 mandatory features

Day 3–4: Design system architecture (Layered Architecture: Controller, Service, Repository, DTO, Security)

Day 5–7: Create component diagram, ERD, and integration architecture

Week 2: Detailed Design

Day 1–2: Define RESTful API specifications with proper status codes
Day 3–4: Database schema design (Users, Items, Requests, Transactions, Messages, Payments)
Day 5–6: UI/UX wireframes (Web + Mobile XML layouts)
Day 7: Final review ensuring compliance with IT342 required features

Phase 2: Backend Development (Week 3-4)

Week 3: Security Core Architecture

Day 1: Spring Boot setup with layered architecture structure

Day 2: Implement JWT authentication and BCrypt password hashing

Day 3: Role-Based Access Control (Admin, Student roles)

Day 4: Protected routes and `/api/user/me` endpoint

Day 5: User registration, login, logout, and profile APIs

Week 4: Core Business & Integrations

Day 1: Core entity CRUD (Items, Requests, Transactions) with validation

Day 2: File upload feature (item image upload stored on server & linked to DB)

Day 3: External Public API integration (e.g., item category suggestions or public dataset display)

Day 4: Google OAuth login integration with JWT generation

Day 5: SMTP Email integration (welcome email + request approval notification)

Phase 3: Web Application (Week 5-6)

Week 5: React Foundation & Authentication

Day 1: React project setup and API service configuration

Day 2: Implement login, register, and OAuth login screens

Day 3: Protected routes using JWT and role-based UI rendering

Day 4: Homepage dashboard consuming real backend APIs

Day 5: Browse items page with search, filter, and request feature

Week 6: Core Modules & Integrations

Day 1: My Items CRUD interface with file upload integration

Day 2: Borrow request management (approve/reject for owner role only)

Day 3: Profile management and role-aware UI restrictions

Day 4: Real-time updates using polling or WebSocket (optional bonus)

Day 5: Full API integration testing (no mock data allowed)

Phase 4: Mobile Application Development (Week 7–8)

Week 7: Android Foundation

Day 1: Android project setup (Kotlin, XML layouts, API Level 34)

Day 2: Implement Login & Register screens using backend JWT

Day 3: Secure JWT storage and Authorization header handling

Day 4: Homepage and navigation drawer with role-aware UI

Day 5: Retrofit integration for protected API endpoints

Week 8: Core Module Consumption

Day 1: Display item catalog and request borrowing via API

Day 2: My Items and Return Items features using protected endpoints

Day 3: Profile screen and role-based UI access control

Day 4: Error handling for API failures and unauthorized responses

Day 5: Testing on emulator/device and final APK generation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: System Integration & Testing

Day 1: End-to-end testing (Web + Mobile + Backend)

Day 2: Verify OAuth login, file upload, and external API display

Day 3: Validate SMTP email sending (welcome & request notifications)

Day 4: Security and role restriction validation (Admin vs Student)

Day 5: Performance optimization and bug fixing

Week 10: Deployment & Finalization

Day 1: Backend deployment (Render / Railway / Cloud VM)

Day 2: Web app deployment (Vercel / Netlify)

Day 3: Mobile APK final build connected to live backend

Day 4: Final system testing with real database and integrations

Day 5: Documentation finalization and defense preparation

Milestones:

- **M1 (End Week 2):** Architecture, ERD, and wireframes completed
- **M2 (End Week 4):** Secure backend with JWT, RBAC, OAuth, Email, and File Upload
- **M3 (End Week 6):** Fully functional React web application with protected routes
- **M4 (End Week 8):** Android mobile app (Kotlin XML + Retrofit) integrated with backend
- **M5 (End Week 10):** Fully integrated system deployed and ready for Final Defense

Critical Path:

1. Authentication & JWT security implementation (Week 3)
2. Role-based access control across API and UI (Week 3–4)
3. Core business module CRUD with validation (Week 4)

4. External API, OAuth login, and SMTP email integration (Week 4–5)
5. Mobile app API integration using Retrofit (Week 7–8)
6. Cross-platform integration and security validation (Week 9)

Risk Mitigation:

- Implement authentication and RBAC first before other modules
- Test API endpoints early using Postman/Swagger
- Use real integrations (OAuth, SMTP, external API) early to avoid late failures
- Maintain version-controlled stable builds for backend, web, and mobile
- Prioritize core CRUD and borrowing workflows before optional features
- Perform continuous integration testing to ensure all required IT342 features work correctly